

Pontificia Universidad Católica del Ecuador

FORMACIÓN DE GRADO — CARRERA DE SOFTWARE

Materia: Desarrollo de Sistemas de Información

Proyecto: PortoSinFiltro - Plataforma web de denuncias ciudadanas para Portoviejo

Estudiante: Johan George Medranda García

Portoviejo - Manabí

Enlace del Repositorio: <https://github.com/castro-bot/portoSinFiltro.git>

Enlace de Producción (Despliegue): <https://porto-sin-filtro.vercel.app/>

Reporte de gestión e historial técnico

Mi aporte se concentró en el desarrollo, pruebas integrales (frontend y backend), aseguramiento de calidad (QA), documentación de contingencia y gestión de base de datos dentro del proyecto **PortoSinFiltro**. A continuación se sintetizan las incidencias y actividades finalizadas en Jira asociadas a mi trabajo:

Jira	Aporte realizado	Evidencia técnica / Detalle
SCRUM-3	Pruebas al Dashboard principal del sistema.	Validación de carga, precisión de métricas y criterios de aceptación visuales.
SCRUM-214	Pruebas de Frontend (UI/UX) para el Dashboard.	Cobertura de componentes, adaptabilidad <i>responsive</i> , estados de carga (<i>loading</i>) y manejo de errores en cliente.



SCRUM-215	Pruebas de Backend para el Dashboard.	Validación de endpoints (src/routes/dashboard.js), integración con Supabase (src/db/supabase.js), middlewares de auth y respuestas HTTP.
SCRUM-16	Refrescar estado tras acciones comunitarias.	Lógica de actualización de estados (ACTIVA, CON AVANCE, RESUELTA) y sincronización tras la interacción del usuario.
SCRUM-11	Base de datos actualizada.	Mantenimiento de esquemas, migraciones SQL (schema.sql, roles, resoluciones) y sincronización del modelo de datos en Supabase.
SCRUM-75	Probar Plan B standalone (sin Supabase/red).	Estrategia integral de contingencia offline. Incluye las subtarefas SCRUM-140 (levantar standalone), SCRUM-141 (flujo crítico) y SCRUM-142 (documentación).
SCRUM-78	Actualizar docs/PLAN-ROLES-COMUNITARIOS.md.	Documentación técnica sobre la asignación y comportamiento de roles (ciudadano vs. administrador) y reglas comunitarias.
SCRUM-80	Formalizar guion de presentación (3-5 min).	Estructuración del flujo del discurso y demostración en vivo de las funcionalidades clave del sistema para la defensa del proyecto.
SCRUM-81	QA manual: recorrer app buscando bugs no listados.	Pruebas exploratorias de extremo a extremo (E2E), verificación de usabilidad y detección de errores limítrofes previo a la demostración.





Historial de código y validación

Durante el ciclo de desarrollo se ejecutó un plan de pruebas en múltiples capas:

1. **Capa Backend y Base de Datos:** Se validó que las peticiones a `src/routes/dashboard.js` apliquen correctamente el middleware `auth.js` y que las consultas a Supabase retornen los tipos de datos requeridos sin exponer errores no controlados.
2. **Capa Frontend y Realtime:** Se aseguraron las transiciones de estado de las denuncias en la interfaz, permitiendo que la interacción del ciudadano (voto de progreso, confirmación de resolución) refresque los contadores y cambie el chip de estado adecuadamente.
3. **Mecanismo de Contingencia (Plan B):** Se verificó la apertura y ejecución independiente de la aplicación en modo local sin red ni backend activo, permitiendo navegar el muro, ver detalles y evaluar la UI mediante datos simulados (*mocks*).
4. **Control de Calidad (QA Exploratorio):** Mediante la ejecución del ticket **SCRUM-81**, se realizó un barrido manual completo para corregir casos de esquina (*edge cases*) en los formularios, filtros del muro y vistas del panel.

Propuestas estratégicas y aprendizaje

Propuesta 1. Plan de Contingencia Standalone (Plan B)

Propuse y estructuré la validación del **Plan B (standalone)** para blindar la presentación del equipo ante eventuales fallas de conexión a internet o de indisponibilidad en los servicios cloud de Supabase. Con esto, el equipo cuenta con una versión funcional local y una guía de conmutación rápida capaz de activarse en menos de un minuto durante la demostración.

Propuesta 2. Estrategia de Pruebas Integrales y Cobertura QA

Propuse abordar la calidad del software desde tres frentes: unitario/integración en API, pruebas de interfaz (UI/UX) y sesiones de QA exploratorio manual. Esta estrategia permitió identificar posibles incoherencias en las transiciones de estado comunitarias y asegurar que el dashboard principal responda correctamente a los permisos y roles configurados.





Conclusión y experiencia personal

La participación en el proyecto **PortoSinFiltro** reforzó mi capacidad para trabajar con arquitecturas web modernas basadas en desacoplamiento (Frontend React/Vite + API Node.js + Backend as a Service Supabase).

El mayor aprendizaje radicó en garantizar la consistencia entre los flujos de usuario, la gestión de incidencias en Jira y el comportamiento real del sistema bajo diferentes escenarios de red. A través de la documentación de procesos, la ejecución de pruebas rigurosas y la planificación de contingencias, logramos consolidar una plataforma robusta, alineada a los requerimientos comunitarios y lista para su defensa académica.

